

## RELATÓRIO TÉCNICO

**Ecossistema de Delivery - Projeto Final** *Desenvolvimento de Banco de Dados Relacional com MySQL 8.0+*

**Disciplina:** Banco de Dados II | **Data:** 27/05/2026

Olívia Lacort Zimmermann

## Sumário

1. Introdução e Objetivo
2. Escopo do Sistema e Requisitos
3. Modelagem Lógica e Física
4. Justificativa das Escolhas Técnicas
5. Objetos de Banco de Dados
  - 5.1 Functions;
  - 5.2 Stored Procedures;
  - 5.3 Trigger de Auditoria;
  - 5.4 View de Relatório Regional;
6. Estrutura de Arquivos do Projeto
7. Considerações Finais

## 1. Introdução e Objetivo

Este relatório documenta o desenvolvimento do Projeto Final da disciplina de Banco de Dados II, cujo objetivo é aplicar, em um cenário de alta complexidade, todos os conceitos de modelagem, normalização e programação em banco de dados, incluindo *Stored Procedures*, *Functions*, *Triggers* e *Views*.

O sistema modelado representa um Ecossistema de Delivery, capaz de gerenciar múltiplos restaurantes, clientes, entregadores, pedidos, cardápios, taxas e atendimento ao cliente (SAC). Toda a implementação foi realizada em MySQL 8.0+, seguindo boas práticas de integridade referencial, desempenho e rastreabilidade de dados.

## 2. Escopo do Sistema e Requisitos

### Requisitos Funcionais (RF)

| Código | Descrição                                         |
|--------|---------------------------------------------------|
| RF01   | Cadastro completo de Usuários (Clientes)          |
| RF02   | Cadastro de Entregadores com dados de veículo     |
| RF03   | Gestão de Restaurantes e Cardápios (Produtos)     |
| RF04   | Controle de Estoque de itens                      |
| RF05   | Processamento de Pedidos e Itens do Pedido        |
| RF06   | Gestão de Taxas (Entrega, Serviço e Cancelamento) |
| RF07   | Sistema de FAQ e Reclamações (SAC)                |
| RF08   | Rastreamento de Status de Entrega                 |

### Requisitos Não Funcionais (RNF)

| Código | Descrição                                                        |
|--------|------------------------------------------------------------------|
| RNF01  | Banco de Dados MySQL 8.0+                                        |
| RNF02  | Senhas armazenadas de forma criptografada (SHA-256)              |
| RNF03  | Integridade referencial garantida por Foreign Keys               |
| RNF04  | Otimização de consultas por região e data via índices            |
| RNF05  | Rastreabilidade de alterações em pedidos via Auditoria (Trigger) |

### 3. Modelagem Lógica e Física

O banco de dados delivery\_db é composto por 9 tabelas, organizadas de forma a atender todos os requisitos funcionais e não funcionais. A seguir, a descrição de cada entidade e seus relacionamentos principais.

| Tabela                   | Descrição                                            | Chave Primária | Chaves Estrangeiras                             |
|--------------------------|------------------------------------------------------|----------------|-------------------------------------------------|
| <b>clientes</b>          | Usuários consumidores do app                         | id_cliente     | —                                               |
| <b>entregadores</b>      | Profissionais de entrega com dados de veículo        | id_entregador  | —                                               |
| <b>restaurantes</b>      | Estabelecimentos parceiros do ecossistema            | id_restaurante | —                                               |
| <b>taxas</b>             | Taxas de entrega, serviço e cancelamento             | id_taxa        | —                                               |
| <b>produtos</b>          | Cardápio dos restaurantes com estoque                | id_produto     | id_restaurante                                  |
| <b>pedidos</b>           | Pedidos realizados pelos clientes                    | id_pedido      | id_cliente,<br>id_restaurante,<br>id_entregador |
| <b>itens_pedido</b>      | Itens individuais de cada pedido                     | id_item        | id_pedido,<br>id_produto                        |
| <b>faq_reclamacoes</b>   | SAC: FAQs, reclamações, elogios e sugestões          | id_faq         | id_cliente,<br>id_pedido                        |
| <b>historico_pedidos</b> | Auditoria imutável de todas as alterações de pedidos | id_historico   | (sem FK intencional)                            |

**Nota sobre historico\_pedidos:** A ausência intencional de *Foreign Keys* nesta tabela garante que o histórico de auditoria seja preservado mesmo que o pedido original seja removido do sistema principal, atendendo perfeitamente ao **RNF05**.

#### Políticas de Integridade Referencial

| Relação                          | ON DELETE | ON UPDATE | Justificativa                 |
|----------------------------------|-----------|-----------|-------------------------------|
| <b>pedidos</b> → <b>clientes</b> | RESTRICT  | CASCADE   | Não permite apagar um cliente |

|                                   |          |         |                                                                                                  |
|-----------------------------------|----------|---------|--------------------------------------------------------------------------------------------------|
| <b>pedidos → restaurantes</b>     | RESTRICT | CASCADE | que possua pedidos realizados. Não permite apagar um restaurante que possua pedidos registrados. |
| <b>pedidos → entregadores</b>     | SET NULL | CASCADE | O registro do pedido pode continuar existindo mesmo que o entregador associado seja removido.    |
| <b>itens_pedido → pedidos</b>     | CASCADE  | CASCADE | Os itens do pedido são deletados automaticamente junto com a exclusão do pedido pai.             |
| <b>itens_pedido → produtos</b>    | RESTRICT | CASCADE | Impede a exclusão de um produto se ele já estiver vinculado a pedidos existentes.                |
| <b>faq_reclamacoes → clientes</b> | SET NULL | CASCADE | Uma dúvida ou FAQ pública pode ter o autor/cliente definido como nulo.                           |
| <b>faq_reclamacoes → pedidos</b>  | SET NULL | CASCADE | Dúvidas gerais de atendimento podem não estar necessariamente                                    |

vinculadas a um pedido específico.

## 4. Justificativa das Escolhas Técnicas

### Tipos de Dados

- **DECIMAL(10,2)**: Utilizado para todos os valores monetários, garantindo precisão financeira absoluta e evitando erros de arredondamento inerentes ao ponto flutuante.
- **ENUM**: Aplicado para campos com estados fixos e previsíveis (como status do pedido, tipo de veículo, tipo de taxa), assegurando a consistência dos dados sem a necessidade de criar tabelas auxiliares complexas.
- **TINYINT(1)**: Utilizado como o padrão nativo do MySQL para representar valores booleanos (flags como "ativo", "disponível", "percentual").
- **VARCHAR(255)**: Definido para o campo senha\_hash para comportar com segurança qualquer hash criptográfico atual ou futuro.
- **CHAR(2)**: Tamanho ideal de armazenamento fixo para as siglas de Unidades Federativas (UF), eliminando desperdício de espaço em disco.

### Índices (RNF04)

Foram criados índices estratégicos para otimizar as consultas mais frequentes e gargalos de busca do sistema, cumprindo o **RNF04**:

| Índice                         | Tabela       | Coluna(s) | Consultas Beneficiadas                                             |
|--------------------------------|--------------|-----------|--------------------------------------------------------------------|
| <b>idx_clientes_regiao</b>     | clientes     | regiao    | Busca rápida de clientes de uma região específica.                 |
| <b>idx_entregadores_status</b> | entregadores | status    | Filtro dinâmico de entregadores disponíveis para novas rotas.      |
| <b>idx_restaurantes_regiao</b> | restaurantes | regiao    | Geração do relatório regional consolidado (vw_relatorio_regional). |
| <b>idx_restaurantes_cidade</b> | restaurantes | cidade    | Busca de estabelecimentos por município.                           |

|                                 |                   |                |                                                                     |
|---------------------------------|-------------------|----------------|---------------------------------------------------------------------|
| <b>idx_produtos_restaurante</b> | produtos          | id_restaurante | Carregamento rápido do cardápio por estabelecimento.                |
| <b>idx_produtos_categoria</b>   | produtos          | categoria      | Filtro rápido de itens por tipo de categoria.                       |
| <b>idx_pedidos_status</b>       | pedidos           | status         | Operação de rastreamento de entregas em tempo real ( <b>RF08</b> ). |
| <b>idx_pedidos_criado_em</b>    | pedidos           | criado_em      | Consultas e filtros de faturamento por períodos.                    |
| <b>idx_historico_registrado</b> | historico_pedidos | registrado_em  | Auditorias cronológicas e busca de logs por data.                   |

### Segurança de Senhas (RNF02)

Todas as senhas são armazenadas como hash seguro via função nativa SHA2(senha, 256) do MySQL. O fluxo de segurança é composto por duas camadas integradas:

1. A função `fn_validarSenha` avalia a complexidade e classifica a senha em **FRACA**, **MEDIA** ou **FORTE** antes de qualquer persistência.
2. A função `fn_encryptarSenha` rejeita automaticamente senhas classificadas como **FRACA**, retornando NULL e impedindo que senhas inseguras avancem.
3. As *Stored Procedures* `sp_cadastrarCliente` e `sp_cadastrarEntregador` consultam `fn_encryptarSenha` e abortam o processo de cadastro imediatamente caso o retorno seja NULL.

## 5. Objetos de Banco de Dados

### 5.1 Functions

Quatro funções determinísticas foram desenvolvidas para garantir a reutilização de regras de validação e formatação na camada de dados:

| Função | Entrada | Saída | Descrição |
|--------|---------|-------|-----------|
|--------|---------|-------|-----------|

|                          |              |              |                                                                                                    |
|--------------------------|--------------|--------------|----------------------------------------------------------------------------------------------------|
| <b>fn_formatarCPF</b>    | VARCHAR(20)  | VARCHAR(14)  | Limpa caracteres extras e formata para 000.000.000-00. Retorna NULL se o valor for inválido.       |
| <b>fn_formatarCNPJ</b>   | VARCHAR(20)  | VARCHAR(18)  | Limpa caracteres extras e formata para 00.000.000/0000-00. Retorna NULL se o valor for inválido.   |
| <b>fn_validarSenha</b>   | VARCHAR(255) | VARCHAR(10)  | Classifica a senha em FRACA, MEDIA ou FORTE de acordo com as regras de complexidade.               |
| <b>fn_encryptarSenha</b> | VARCHAR(255) | VARCHAR(255) | Valida a complexidade da senha e retorna o hash SHA-256 correspondente. Retorna NULL se for FRACA. |

*Critérios de classificação de complexidade de fn\_validarSenha:*

- **FRACA:** Menos de 8 caracteres. É rejeitada por padrão pela função de encriptação.
- **MEDIA:** Possui 8 ou mais caracteres, mas carece de uma combinação completa de maiúsculas, minúsculas, números ou símbolos.
- **FORTE:** Possui 8 ou mais caracteres, contendo obrigatoriamente letras maiúsculas, letras minúsculas, números e pelo menos um caractere especial.

**Stored Procedures**

| Procedure | Parâmetros IN principais | Validações Realizadas |
|-----------|--------------------------|-----------------------|
|-----------|--------------------------|-----------------------|



|                               |                                                     |                                                                                                                                               |
|-------------------------------|-----------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| <b>sp_cadastrarCliente</b>    | nome, email, cpf, senha, endereco                   | Formata o CPF; valida e encripta a senha; bloqueia cadastros com e-mail ou CPF duplicados.                                                    |
| <b>sp_cadastrarEntregador</b> | nome, cpf, cnh, tipo_veiculo, cnpj, senha           | Formata o CPF e o CNPJ; valida e encripta a senha; bloqueia duplicidade de CPF e CNH.                                                         |
| <b>sp_cadastrarProduto</b>    | id_restaurante, nome, categoria, preco              | Verifica se o restaurante associado está ativo; valida se o preço é \$\ge 0\$; obriga o preenchimento da categoria.                           |
| <b>sp_cadastrarPedido</b>     | id_cliente, id_restaurante, itens ( <i>string</i> ) | Valida o estoque de cada produto; calcula os totais com taxas; atualiza o estoque de forma atômica utilizando transações (START TRANSACTION). |

A `sp_cadastrarPedido` é o núcleo transacional do sistema. Ela recebe os itens em lote no formato string contendo 'id\_produto:quantidade,id\_produto:quantidade', processa a informação usando um laço de repetição (WHILE LOOP), valida as quantidades físicas disponíveis, desconta o estoque de cada produto de forma consistente e consolida os valores globais sob o controle estrito de START TRANSACTION / COMMIT / ROLLBACK.

#### Trigger de Auditoria

Dois triggers foram criados na tabela de pedidos para garantir a rastreabilidade exigida pelo **RNF05**:

| Trigger                           | Ação                    | Evento / Comportamento                               |
|-----------------------------------|-------------------------|------------------------------------------------------|
| <b>tg_auditoria_pedido_insert</b> | AFTER INSERT em pedidos | Registra um espelho exato do pedido criado na tabela |

historico\_pedidos com a operação 'INSERT'.

|                                   |                         |                                                                                                         |
|-----------------------------------|-------------------------|---------------------------------------------------------------------------------------------------------|
| <b>tg_auditoria_pedido_update</b> | AFTER UPDATE em pedidos | Salva o novo estado atualizado do pedido na tabela historico_pedidos marcando a operação como 'UPDATE'. |
|-----------------------------------|-------------------------|---------------------------------------------------------------------------------------------------------|

O uso do momento **AFTER** (e não BEFORE) assegura que o registro de auditoria ocorra apenas após a confirmação lógica da operação no banco de dados. Dessa forma, cada transição no ciclo de vida de um pedido (por exemplo: *pendente* → *confirmado* → *em\_preparo* → *saiu\_entrega* → *entregue*) gera uma nova entrada imutável no log histórico.

### View de Relatório Regional

A view vw\_relatorio\_regional consolida dados estruturados de 5 tabelas (pedidos, clientes, restaurantes, itens\_pedido e produtos) através de junções eficientes com a tabela entregadores (LEFT JOIN), expondo as seguintes consultas:

| Filtro                | Coluna na View                        | Exemplo Prático de Uso                           |
|-----------------------|---------------------------------------|--------------------------------------------------|
| Região do Restaurante | regiao_restaurante                    | WHERE<br>regiao_restaurante =<br>'Sudeste'       |
| Período               | ano_pedido, mes_pedido,<br>dia_pedido | WHERE ano_pedido =<br>2025 AND mes_pedido =<br>5 |
| Tipo de Item          | tipo_item ( <i>categoria</i> )        | WHERE tipo_item =<br>'pizza'                     |
| Valor Total           | valor_total                           | WHERE valor_total ><br>80.00                     |

As colunas de granularidade de tempo (ano\_pedido, mes\_pedido e dia\_pedido) foram separadas propositalmente na View para evitar o uso de funções diretas de manipulação de data no filtro WHERE, o que prejudicaria o uso do índice idx\_pedidos\_criado\_em (RNF04).

## 6. Estrutura de Arquivos do Projeto

O projeto está organizado na árvore de diretórios padrão de entrega:

|                |                           |
|----------------|---------------------------|
| <b>Arquivo</b> | <b>Conteúdo do Script</b> |
|----------------|---------------------------|

|                               |                                                                                                                                  |
|-------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| /scripts/01_ddl_schema.sql    | Estrutura de criação de tabelas, índices físicos e constraints (DDL)                                                             |
| /scripts/02_dml_inserts.sql   | Carga de dados mínimos de teste (ao menos 20 inserts para cada uma das 9 tabelas)                                                |
| /scripts/03_functions.sql     | Código fonte das 4 funções:<br>fn_formatarCPF, fn_formatarCNPJ,<br>fn_validarSenha, fn_encryptarSenha                            |
| /scripts/04_procedures.sql    | Implementação das 4 procedures:<br>sp_cadastrarCliente,<br>sp_cadastrarEntregador,<br>sp_cadastrarProduto,<br>sp_cadastrarPedido |
| /scripts/05_triggers.sql      | Implementação dos 2 triggers de auditoria: tg_auditoria_pedido_insert e tg_auditoria_pedido_update                               |
| /scripts/06_views_queries.sql | Criação da view vw_relatorio_regional + 6 queries analíticas + 5 updates/deletes de teste                                        |
| /docs/relatorio_tecnico.pdf   | Versão em PDF deste relatório técnico                                                                                            |
| /README.md                    | Guia explicativo de instalação com link para o vídeo de demonstração                                                             |

#### Ordem Recomendada de Execução dos Scripts:

1. 01\_ddl\_schema.sql (Criação da base e tabelas)
2. 02\_dml\_inserts.sql (Alimentação inicial da base)
3. 03\_functions.sql (Criação de funções essenciais)
4. 04\_procedures.sql (Criação de procedimentos de escrita e transações)
5. 05\_triggers.sql (Criação dos gatilhos de rastreamento de auditoria)
6. 06\_views\_queries.sql (Criação de views e testes gerais)

## 7. Considerações Finais

O projeto final desenvolvido atendeu integralmente a todos os requisitos solicitados na disciplina de Banco de Dados II. A seguir, apresenta-se o quadro consolidado com o status dos objetos projetados e entregues:

| <b>Objeto de Banco</b>                 | <b>Quantidade Proposta</b> | <b>Status de Entrega</b> |
|----------------------------------------|----------------------------|--------------------------|
| <b>Tabelas (DDL)</b>                   | 9                          | ✓ Concluído              |
| <b>Índices</b>                         | 15                         | ✓ Concluído              |
| <b>Foreign Keys</b>                    | 7                          | ✓ Concluído              |
| <b>Inserts por Tabela</b>              | 20+                        | ✓ Concluído              |
| <b>Updates de Teste</b>                | 5                          | ✓ Concluído              |
| <b>Deletes de Teste</b>                | 5                          | ✓ Concluído              |
| <b>Functions</b>                       | 4                          | ✓ Concluído              |
| <b>Stored Procedures</b>               | 4                          | ✓ Concluído              |
| <b>Triggers</b>                        | 2                          | ✓ Concluído              |
| <b>Views</b>                           | 1                          | ✓ Concluído              |
| <b>Queries Analíticas sobre a View</b> | 6                          | ✓ Concluído              |

Este ecossistema demonstra de maneira prática que um modelo físico de dados robusto e bem parametrizado é capaz de assegurar políticas comerciais e regras de negócio complexas diretamente na base, garantindo a consistência transacional e a alta performance de leitura e escrita do banco de dados independentemente do cliente de conexão.